# IY3660 – Applications of Cryptography

## Week 12 Lecture
## Intro to TLS

Elizabeth Quaglia
Information Security Group

# In today's lecture

<u>Case study: the TLS protocol</u>

Putting it all together!

- Overview of TLS
- TLS Record Protocol
- TLS Handshake Protocol

# Overview of TLS

# History

- SSL = Secure Sockets Layer
  - Developed by Netscape in mid 1990s
  - SSLv1 broken at birth
  - SSLv2 flawed in several ways, now IETF-deprecated (RFC 6176)
  - SSLv3 now considered broken (POODLE + RC4 attacks), but still widely supported

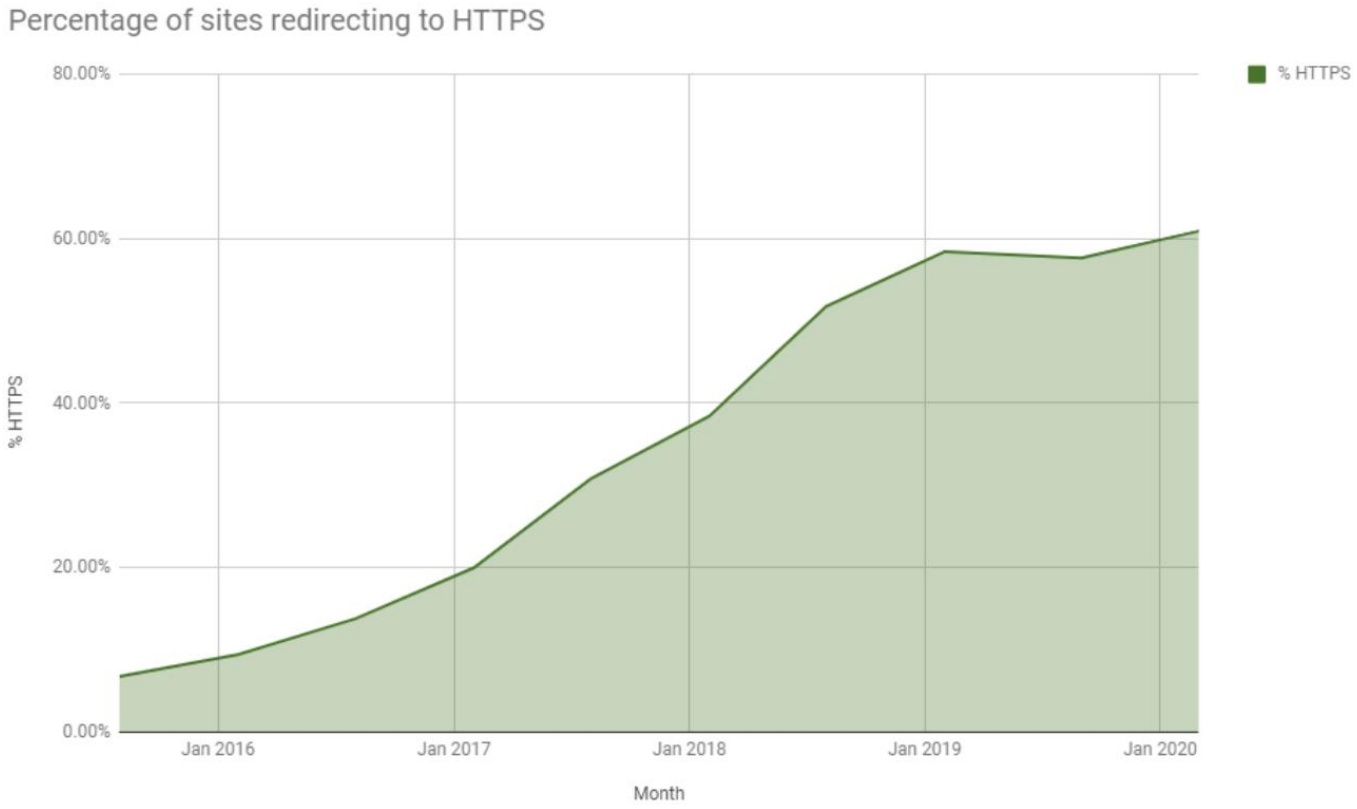
- TLS = Transport Layer Security
  - IETF-standardised version of SSL
  - TLS 1.0 in RFC 2246 (1999)
  - TLS 1.1 in RFC 4346 (2006)
  - TLS 1.2 in RFC 5246 (2008)
  - TLS 1.3 in RFC 8446 (2018)

# Importance of TLS

- Originally designed for secure e-commerce: *https://www.mysite.com*
- Now used much more widely
  - Retail customer access to online banking facilities
  - Access to gmail, facebook, X, etc
  - Mobile applications, including banking apps
  - Payment infrastructures
- TLS has become a de facto secure protocol of choice
  - Used by hundreds of millions (billions?) of people and devices every day
- It is therefore a highly important protocol

# Usage trend of TLS

- By Mar 2020, just over 3 out of 5 of Alexa Top 1 Million sites were using TLS (up from ~7% in 2015!)




| Month | % HTTPS |
|---|---|
| Mar 2020 | 60.93% |
| Sep 2019 | 57.66% |
| Feb 2019 | 58.44% |
| Aug 2018 | 51.78% |
| Feb 2018 | 38.42% |
| Aug 2017 | 30.78% |
| Feb 2017 | 19.96% |
| Aug 2016 | 13.76% |
| Feb 2016 | 9.39% |
| Aug 2015 | 6.71% |

Source: https://scotthelme.co.uk/top-1-million-analysis-march-2020/

# TLS: network features

- TLS follows the client/server paradigm
  - Different roles played by the client and the server
- TLS sits on top of the Transport Layer
  - Layer 5 in the OSI model
- TLS requires a reliable transport protocol
  - Mainly used over TCP
  - RFC 6347 defines TLS over UDP (DTLS)
- TLS is application protocol-independent
  - Many applications can transparently use TLS
    - e.g., HTTP, FTP, Telnet, etc

# TLS: crypto features

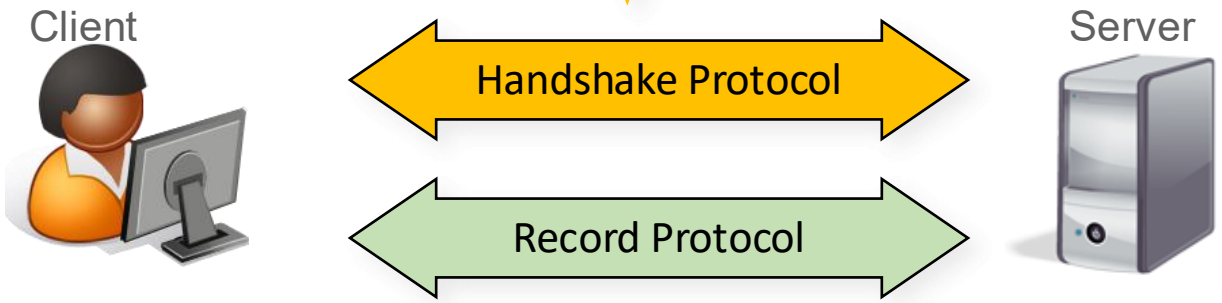
- RFC 5246 (TLS 1.2) stated the protocol goal as:
  - The primary goal is to **provide privacy** and **data integrity** between two communicating **applications**
- RFC 8446 (TLS 1.3) stated the goal as:
  - The primary goal is to provide a **secure channel** between **two communicating peers**
  - The only requirement from the underlying transport is a reliable, in-order data stream
- The TLS secure channel should provide:
  - Confidentiality
  - Authentication
  - Integrity
- Note: non-repudiation is not a goal!

# TLS: crypto features

- TLS is a **hybrid** cryptographic protocol:
- **Public-key cryptography** is used for
  - Peer authentication
  - Key establishment
- **Symmetric-key cryptography** is used
  - To provide data confidentiality and integrity
    - Either via encryption and MACing
    - Or (more commonly) by means of a secure AE algorithm

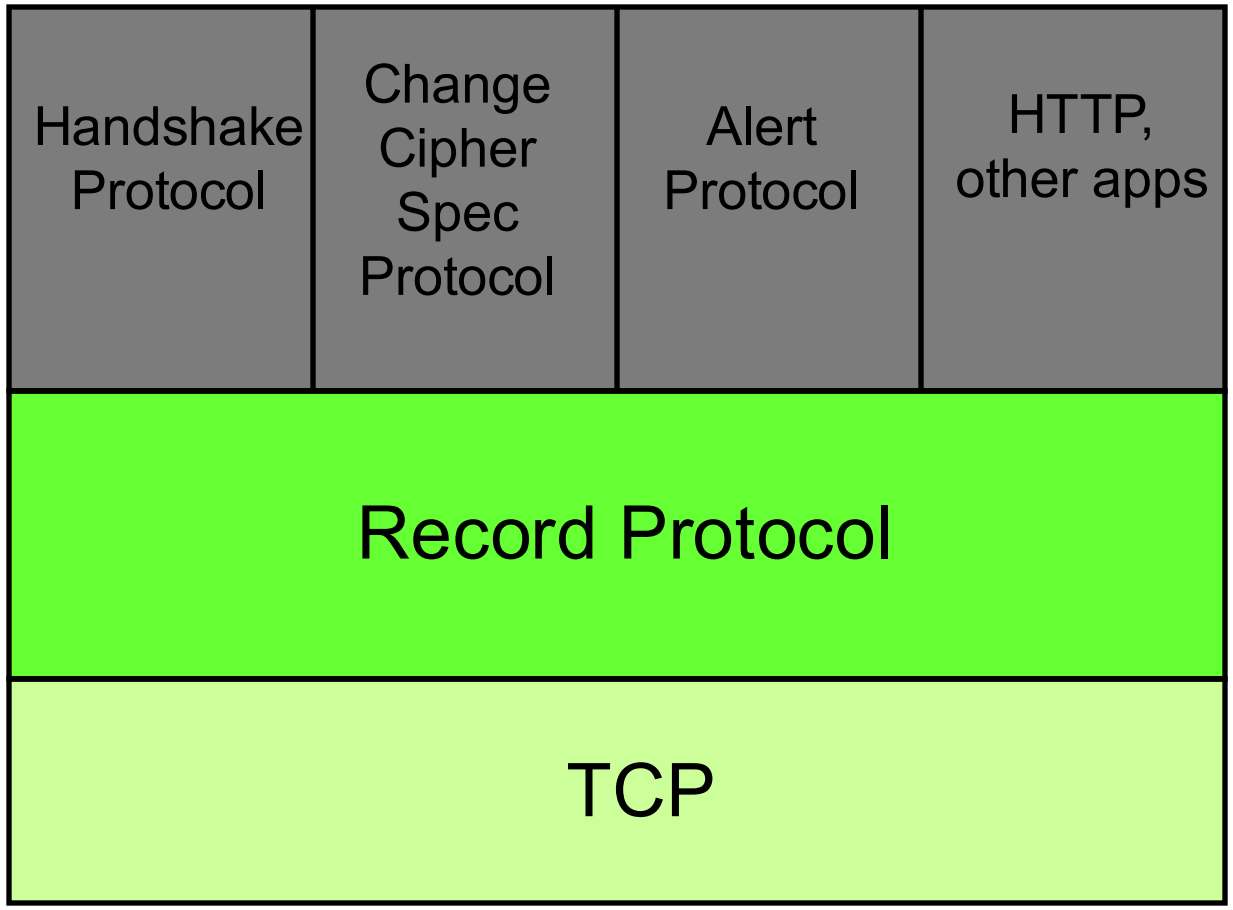
# Highly simplified view of TLS

Used by client and server to
1. Negotiate cipher suite
2. Authenticate (the *communicating peers*)
3. Establish keys used in the Record Protocol




Provides confidentiality and integrity/authenticity of application layer data using keys produced during the Handshake Protocol

# TLS protocol architecture



# The TLS ecosystem (1/3)

- **Servers**
  - Including managed service providers (CloudFlare, Akamai)
- **Clients**
  - Of all shapes and sizes
  - Web browsers to embedded devices
- **Certification Authorities (CAs)**
  - Of all shapes, sizes and levels of security
  - 300+ root CA keys in a typical browser
- **Software vendors**
  - From Google (BoringSSL) down to individuals
  - OpenSSL used by approx 90% of web servers
- **Hardware vendors**

# The TLS ecosystem (2/3)

- TLS versions:
  - SSL 3.0, TLS 1.0, TLS 1.1, TLS 1.2, TLS 1.3
  - Some servers even still support SSL 2.0
- Hundreds of cipher suites*
  - Some very common
    - `TLS_RSA_WITH_AES_128_CBC_SHA256`
    - `TLS_ECDHE_RSA_WITH_AES_128_GCM_SHA256`
  - Some highly esoteric
    - `TLS_KRB5_WITH_3DES_EDE_CBC_MD5`
  - Some offering no security
    - `TLS_NULL_WITH_NULL_NULL !`

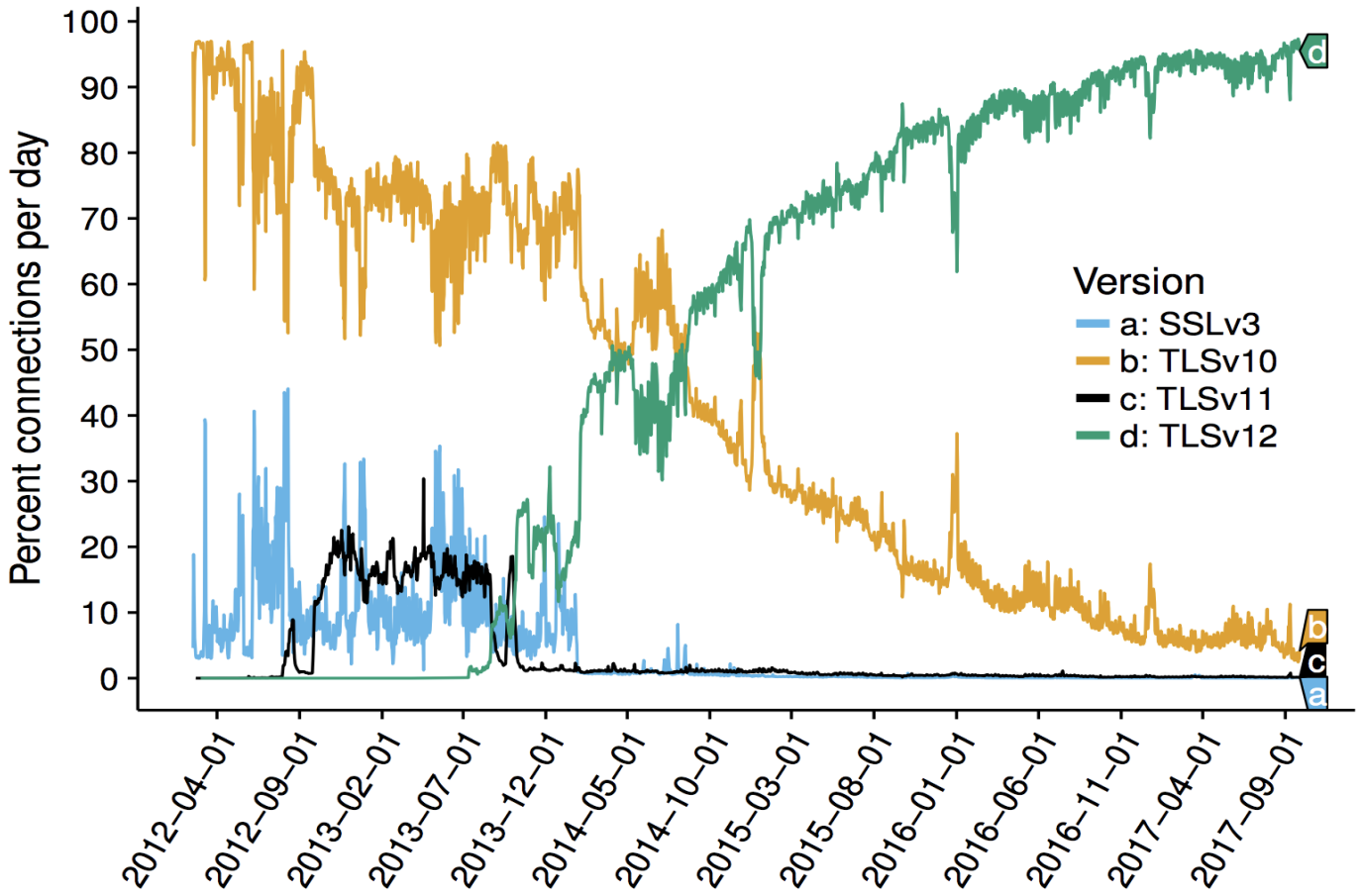
* https://ciphersuite.info/cs/
https://en.wikipedia.org/wiki/Cipher_suite

# The TLS ecosystem (3/3)

- IETF TLS Working Group
- IETF UTA Working Group
  - UTA = Using TLS in Applications
- CFRG (Crypto Forum Research Group)

- Growing community of researchers
  - Finding attacks and building security proofs
  - Analysis of TLS 1.3 drafts
  - Contribution to RFCs

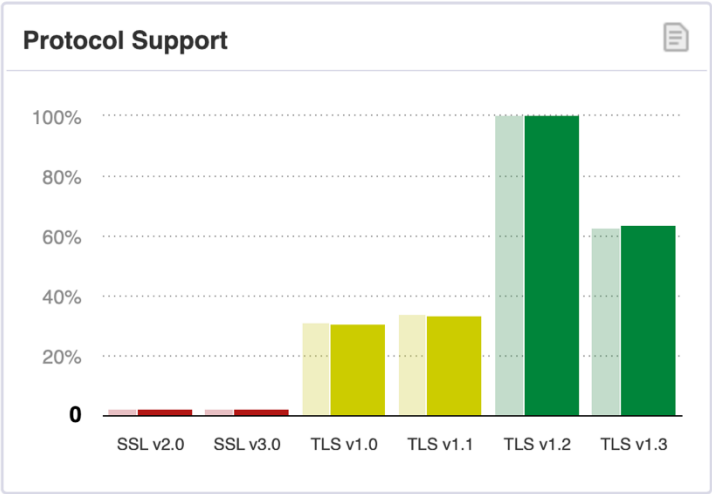
# SSL/TLS versions in use on the internet (2017)




Source: Amann et al. “Mission Accomplished? HTTPS Security after DigiNotar”, IMC 2017

# TLS versions support

- In March 2021 TLS versions 1.0 and 1.1 were deprecated by IETF
- In 2025: **99.9%** of the Alexa top 150K websites support **TLS 1.2,** and **63.5%** support **TLS 1.3**



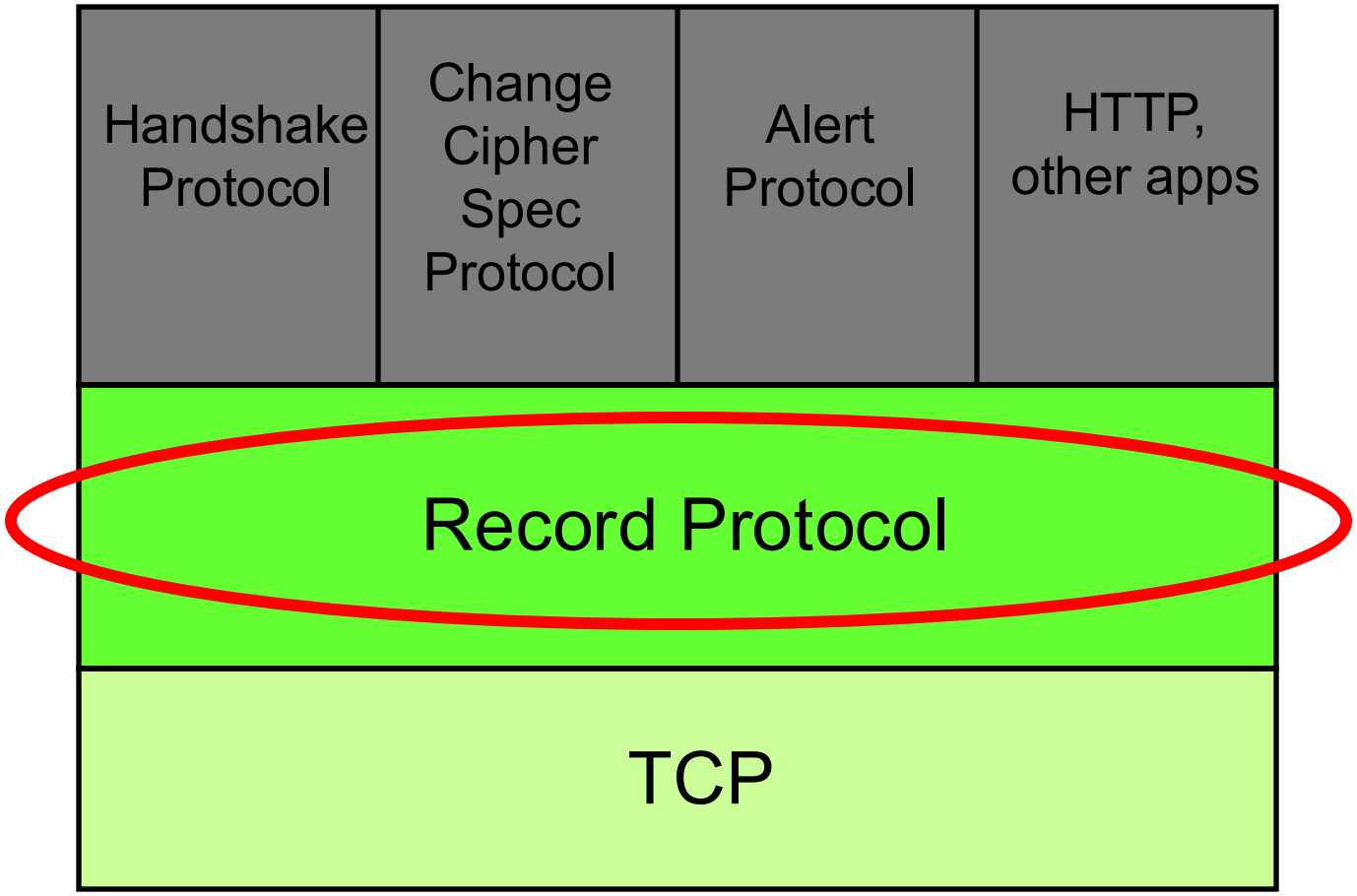

Source: ssl pulse, accessed Dec 2025
https://www.ssllabs.com/ssl-pulse/

TLS 1.3 support in most major browsers:

- since Chrome 70 (2018)
- since Firefox 63 (2018)
- since Safari 14 (2020)
- since Edge 79 (2020)

# TLS Record Protocol (details are from TLS 1.2)

# TLS architecture



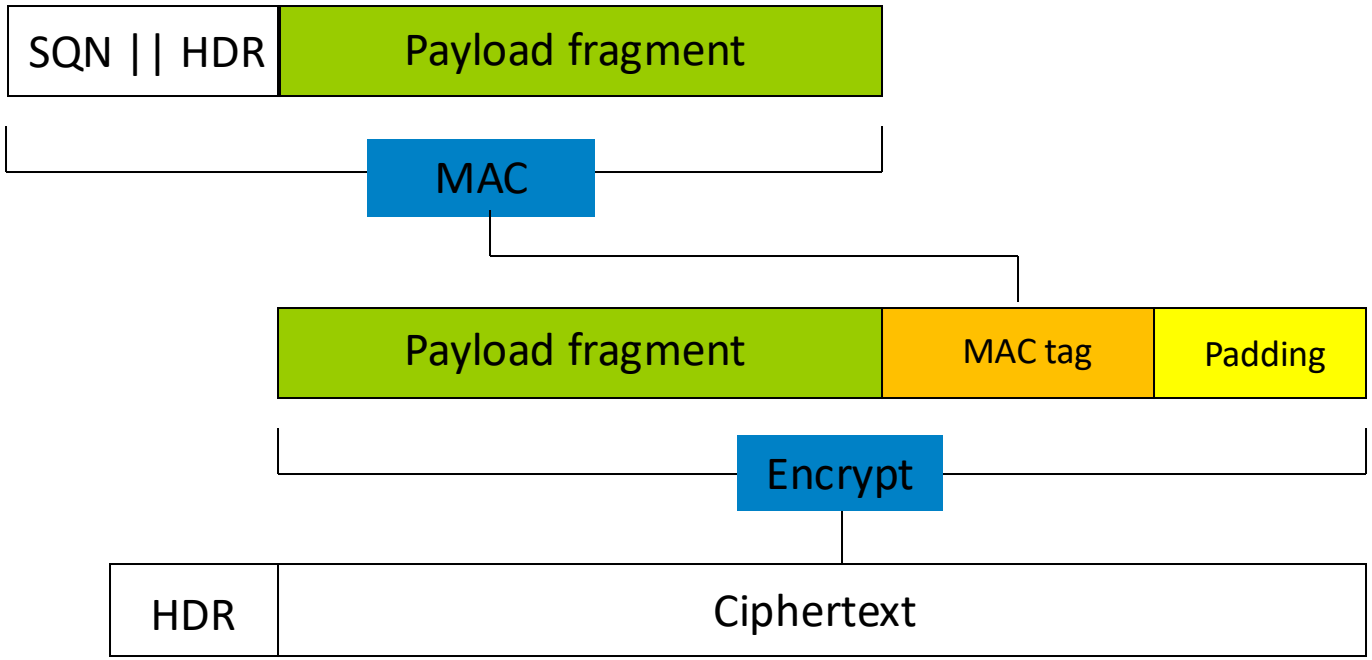
# TLS Record Protocol

- TLS v1.2 Record protocol provides
  - Data origin authentication, integrity using a MAC
  - Confidentiality using a symmetric encryption algorithm
  - Anti-replay using sequence numbers protected by the MAC
- TLS v 1.3 only uses AEAD for confidentiality and integrity

# Operation of TLS Record Protocol

- Data from layer above is received and partitioned into fragments (max size $2^{14}$ bytes)
- Optional data compression
  - Default option is no compression
- Calculate MAC on sequence number, header fields, and data, and append MAC to data
- Pad (if needed by encryption mode), then encrypt
- Prepend 5-byte header, containing
  - Content type (1 byte)
    - Indicating content of record, e.g., handshake message, application message, etc
  - SSL/TLS version (2 bytes)
  - Length of fragment (2 bytes)
- Submit to TCP

# TLS Record Protocol: MAC-Encode-Encrypt




| | |
|---|---|
| MAC | HMAC-MD5, HMAC-SHA1, HMAC-SHA256 |
| Encrypt | CBC-AES128, CBC-AES256, CBC-3DES, RC4-128 |
| Padding | “00” or “01 01” or “02 02 02” or …. or “FF FF….FF” |

# Operation of TLS Record Protocol

- In-bound processing steps reverses these steps:
  1. Receive message, of length specified in HDR
  2. Decrypt
  3. Remove padding
  4. Check MAC
  5. (Optionally: decompress payload)
  6. Pass payload to upper layer
- Note: no defragmentation
  - TLS just provides a stream of fragments to the application
- Errors can arise from decryption, padding removal, or MAC checking steps
- All are fatal errors: error message is sent, and connection is terminated

# TLS Record Protocol Sequence Numbers

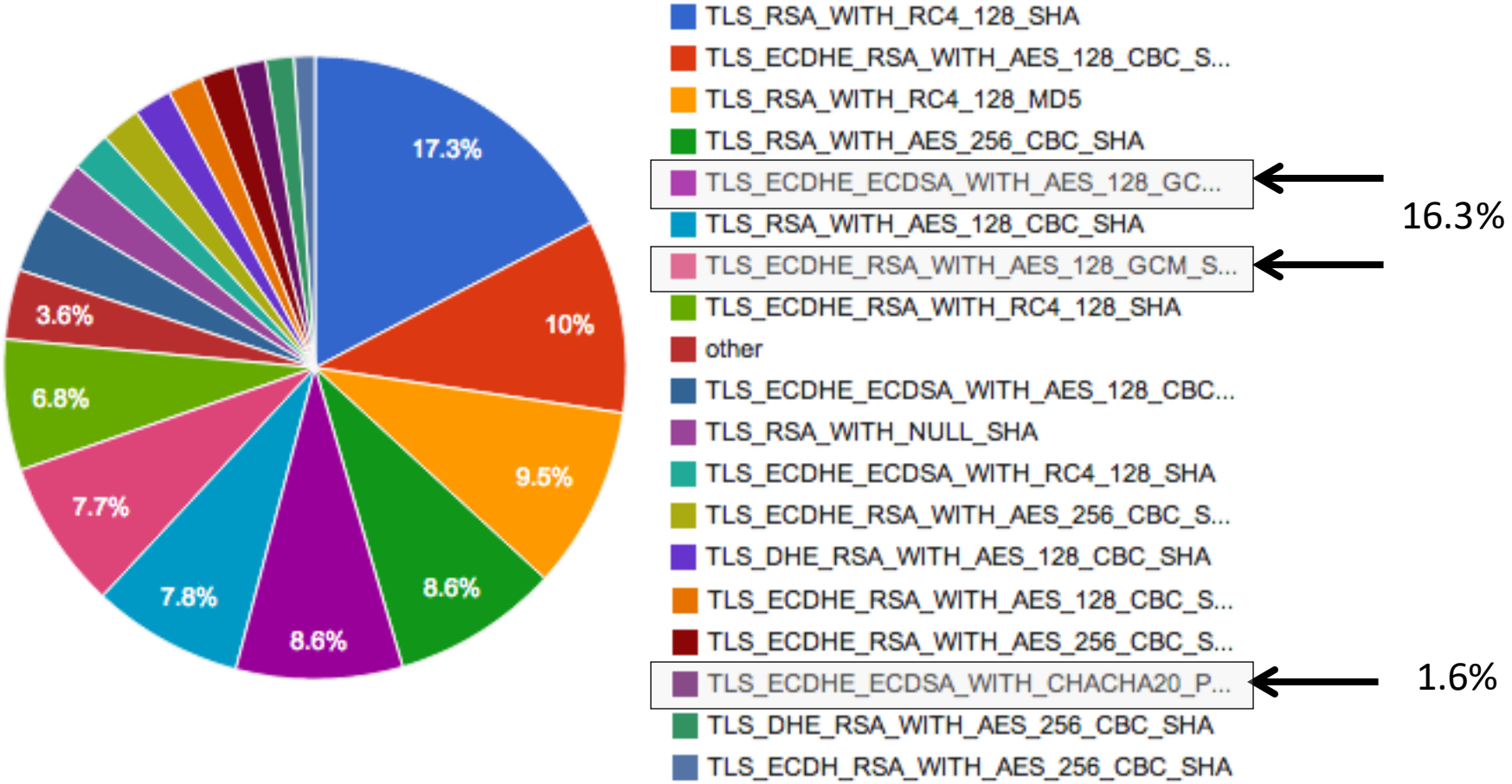
- Sequence number SQN is 64 bits in size and is incremented for each new message
- SQN not transmitted as part of message
  - Each party maintains its own view of the current value of SQN
  - TLS is reliant on TCP to deliver messages in order
- Use of wrong sequence number at receiver leads to failure of MAC verification
  - A fatal error leading to TLS connection termination
- Creates stateful encryption scheme
  - Preventing replay, insertion, reordering attacks,…

# AEAD and TLS Record Protocol

- Dedicated Authenticated Encryption with Associated Data (AEAD) algorithms are supported in TLS 1.2
  - In addition to MAC- then-Encode-then-Encrypt (MEE)
- Single algorithm providing both confidentiality and integrity/data origin
- General AEAD interface specified in RFC 5116 (2008)
- AES-GCM specified in RFC 5288
- AES-CCM specified in RFC 6655
- ChaCha20-Poly1305 in RFC 7905

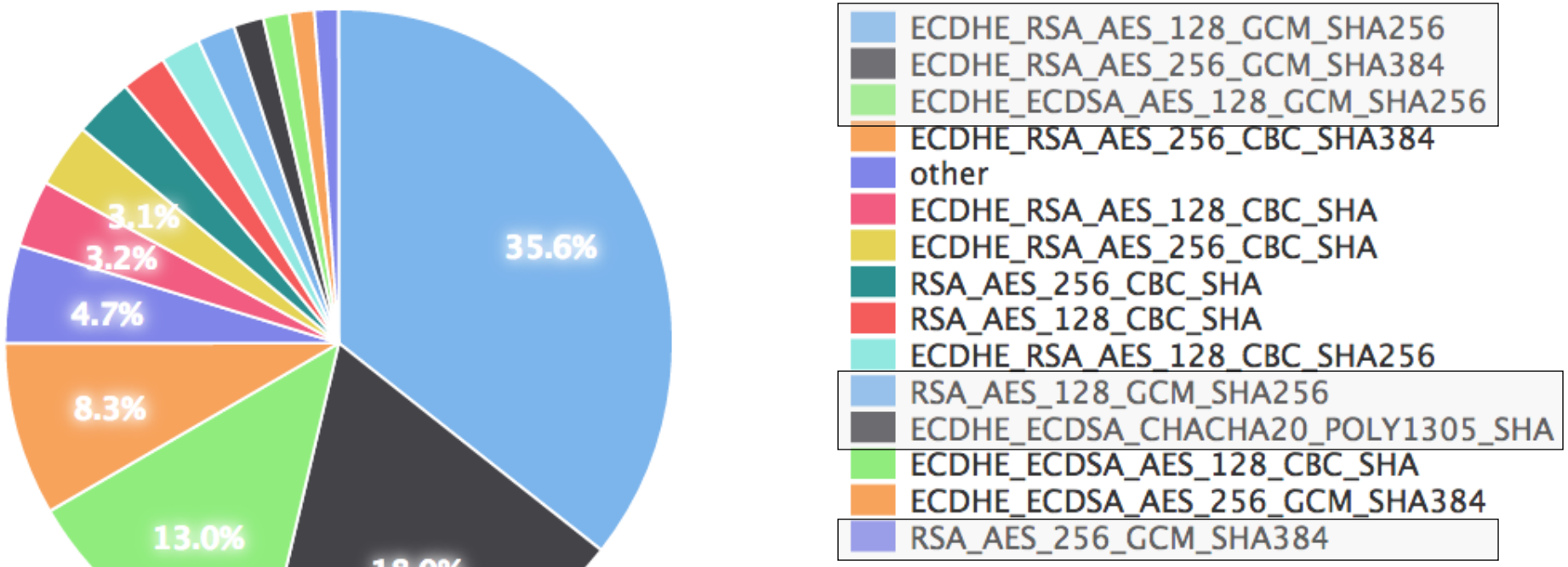
# AEAD Usage in TLS (Sep 2014)

- Snapshot from ICSI Certificate Notary Project



# AEAD Usage in TLS (Dec 2016)

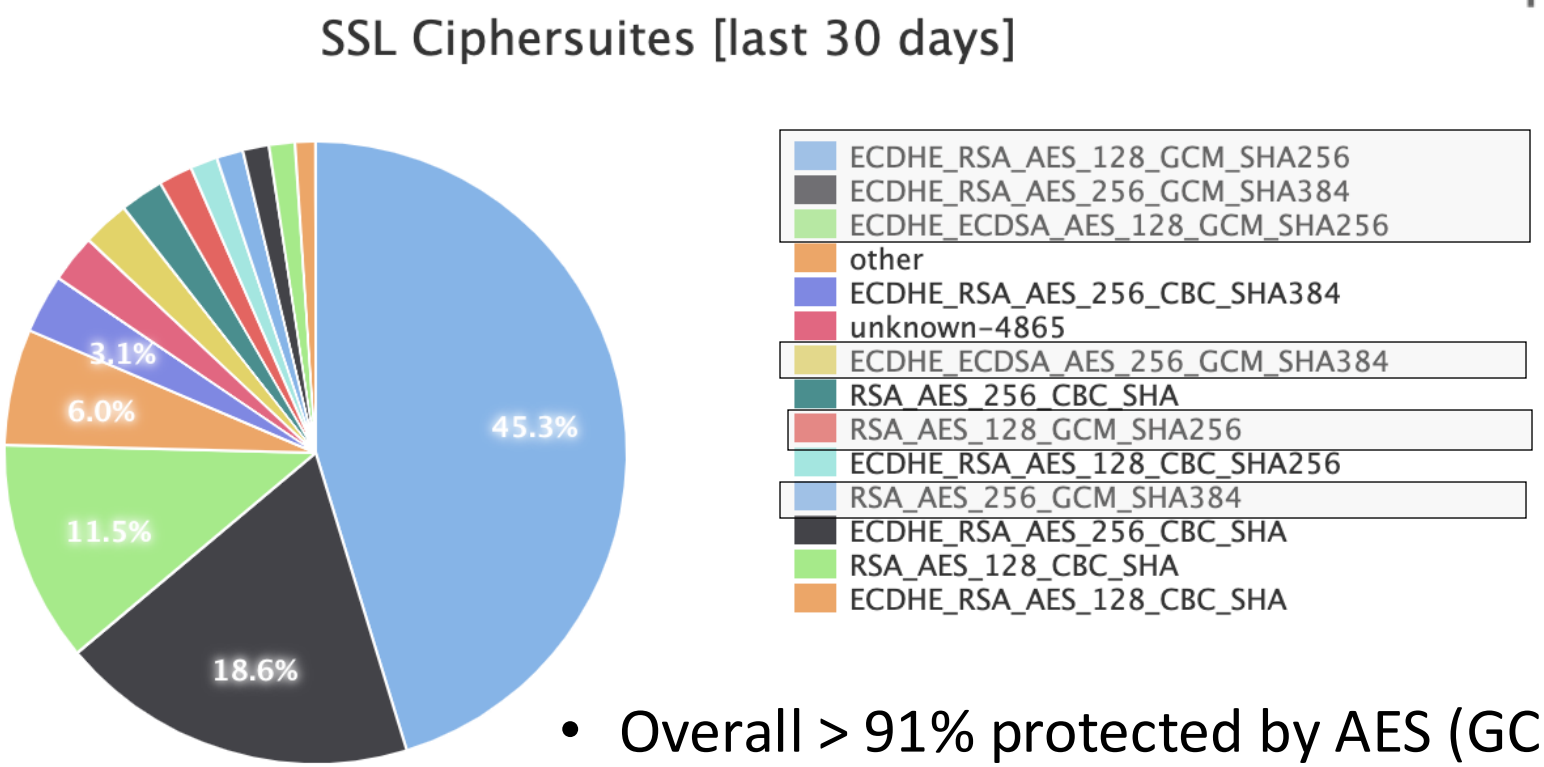
- Snapshot from ICSI Certificate Notary Project:




Total of AES-GCM above 66%

# AEAD Usage in TLS (Dec 2021)

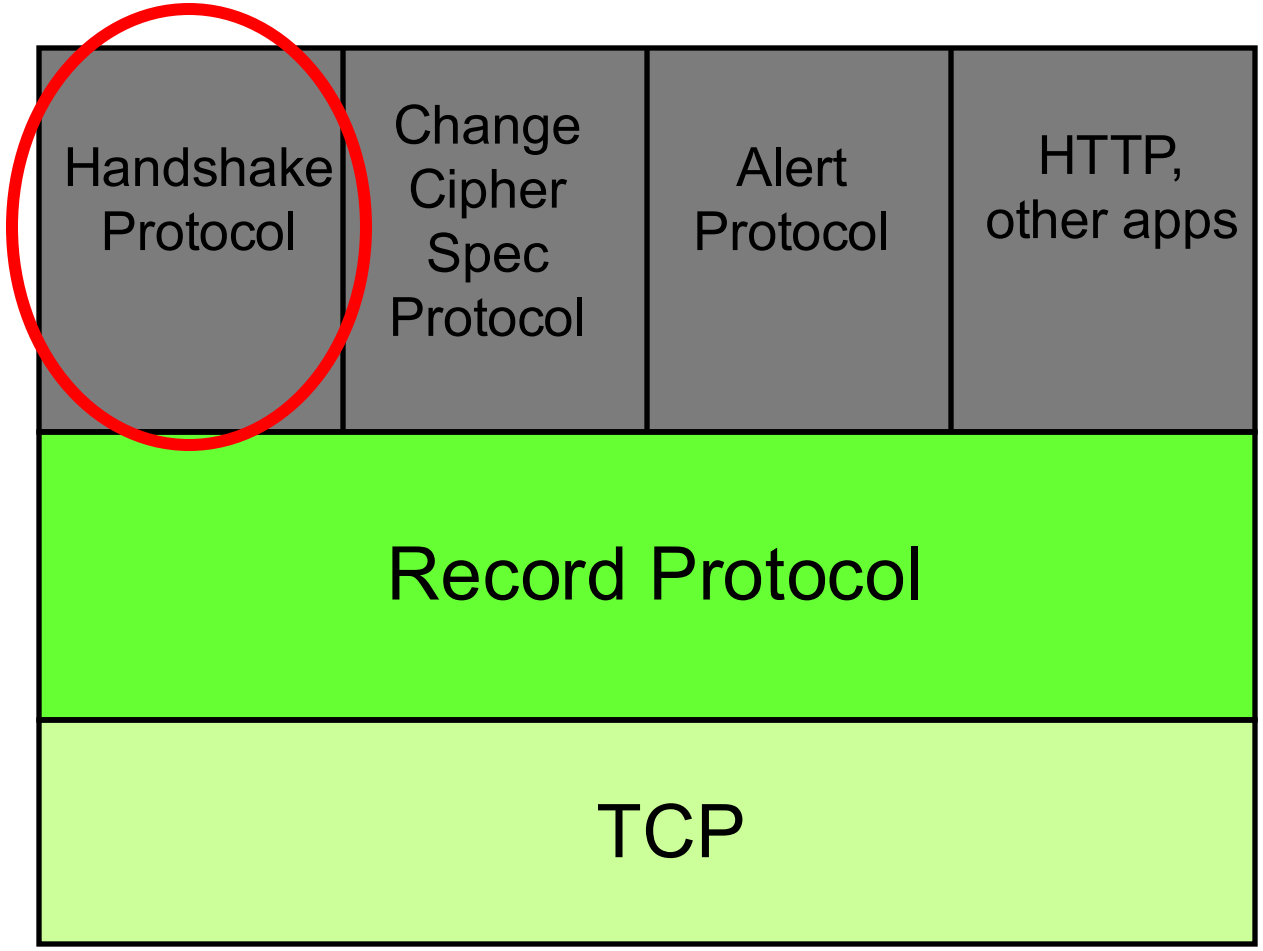
- Snapshot from ICSI Certificate Notary Project




- Overall > 91% protected by AES (GCM or CBC)
- Almost no dedicated stream cipher (ChaCha or RC4!)

# TLS Handshake Protocol (details from TLS 1.2)

# TLS protocol architecture



# Goals of the Handshake Protocol

- Establishes keys (and IVs) needed by the Record Protocol
  - Via establishment of the TLS `master_secret` and subsequent key derivation
- Provides authentication of server (usually) and client (rarely)
  - Using public-key cryptography supported by digital certificates
  - Or pre-shared key (less commonly)
  - Note: In TLS 1.3 the server is **always** authenticated
- Protects negotiation of all cryptographic parameters
  - SSL/TLS version number
  - Encryption and hash algorithms
  - Authentication and key establishment methods
  - To prevent version rollback and cipher suite downgrade attacks

# Key establishment

- TLS supports several key establishment mechanisms
- Method used is negotiated during the Handshake Protocol itself
  - Client sends list of cipher suites it supports in `ClientHello`
    - e.g., `TLS_RSA_WITH_AES_256_CBC_SHA256`
    - e.g., `TLS_ECDHE_RSA_WITH_RC4_128_SHA`
  - Server selects one and tells client in `ServerHello`
- A common choice was RSA encryption (up to TLS 1.2)
  - Server sends RSA public key and certificate chain after ServerHello
  - Client (randomly) generates pre_master_secret, encrypts using public RSA key of server, sends to server
  - RSA encryption is based on PKCS#1 v1.5 padding method

# RSA-based key establishment (simplified)

**Client** **Server**

ClientHello (TLS_RSA_WITH_AES_256_CBC_SHA256)

→

ServerHello, Cert, ServerHelloDone

←

1. Check Cert
2. Extract PubK from Cert
3. Select random pre_master_secret
4. Compute $Enc_{PubK}$(pre_master_secret)

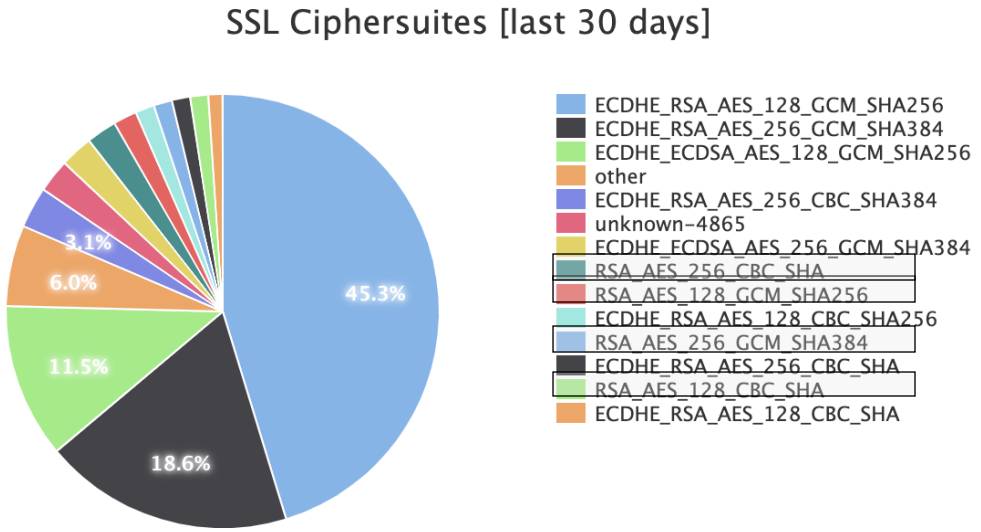
ClientKeyExchange: $Enc_{PubK}$(pre_master_secret)

→

Decrypt to find pre_master_secret

# RSA key exchange not supported in TLS 1.3…

- RSA key exchange is vulnerable to some attacks
  - e.g., Bleichenbacher's million-message attack
- RSA key exchange does not offer forward secrecy
- Indeed, it is now rarely used




Source: https://notary.icsi.berkeley.edu/ (Dec 2021)

# Ephemeral DH-based key establishment (simplified)

**Client** | **Server**

ClientHello (TLS_DHE_RSA_WITH_AES_128_GCM_SHA256)

→

ServerHello, Cert, ServerKeyExchange, ServerHelloDone

←

$p$, $g$, $g^x$,
RSAsig(nonces, params)

1. Check Cert
2. Extract PubK from Cert
3. Use PubK to verify server signature
4. Select random $y$, compute $g^y$, $(g^x)^y$

ClientKeyExchange: $g^y$

→

**pre_master_secret:** $(g^y)^x$

**Note**: in the example, RSA is used to **sign** the DH parameters, not to encrypt data (as it is the case for RSA-based key establishment). One could use other signature schemes (e.g., DSA)

# Key establishment notes

- Typical `ClientHello` offers many different cipher suites
- Choice of which to use is made by server
- `ClientHello` and `ServerHello` contain 32-byte nonces
  - 28-byte random values + 4-byte time encoding
- Nonces signed by the server in DH-based suites, and involved in key derivation
- `ClientHello` also offers SSL/TLS version number
  - Server replies with its choice
- Semantics:

Client: I support up to version x

Server: we will use version y ≤ x

# Forward Secrecy

- Property offered by key establishment protocols
- The compromise of long-term private keys does not compromise the secrecy of past sessions

<u>Example</u>

- Suppose a webserver has a long-term RSA encryption key pair (used in the TLS RSA-based key establishment protocol)
- The long-term public key is used to encrypt the `pre_master_secret` by clients during the handshake, which is then decrypted by the server using the corresponding long-term RSA private key
- This `pre_master_secret` plus public data (in the handshake) will be used to derive the session keys (used in the TLS record protocol to protect data exchanged between client and server)

# Forward Secrecy

- TLS RSA-based key exchange does **not** offer forward secrecy!

<u>Attack:</u>

- Assume an adversary records the data of TLS handshake and record protocol data exchanged between the server and clients for time $t < T$
- Assume at time $t = T$, the attacker recovers the server's RSA private key
- Then the attacker can go back and recover the `pre_master_secret` for all recorded sessions at time $t < T$ by decrypting the handshake's `ClientKeyExchange` messages = $\mathtt{Enc}_{\mathtt{PubK}}$(`pre_master_secret`)
- They can then use `pre_master_secret` to derive the corresponding session keys
- The adversary can now read all messages exchanged by server before time $T$

# Forward Secrecy

- An attacker who learns the RSA private key can decrypt old sessions
- They can also passively eavesdrop on all future RSA-based sessions!
- Also applies to (Static) Diffie-Hellman
  - If attacker recovers long term secret x, it can compute $g^{yx}$ for all sessions.
- A well-known issue given renewed prominence by the Snowden revelations in 2013

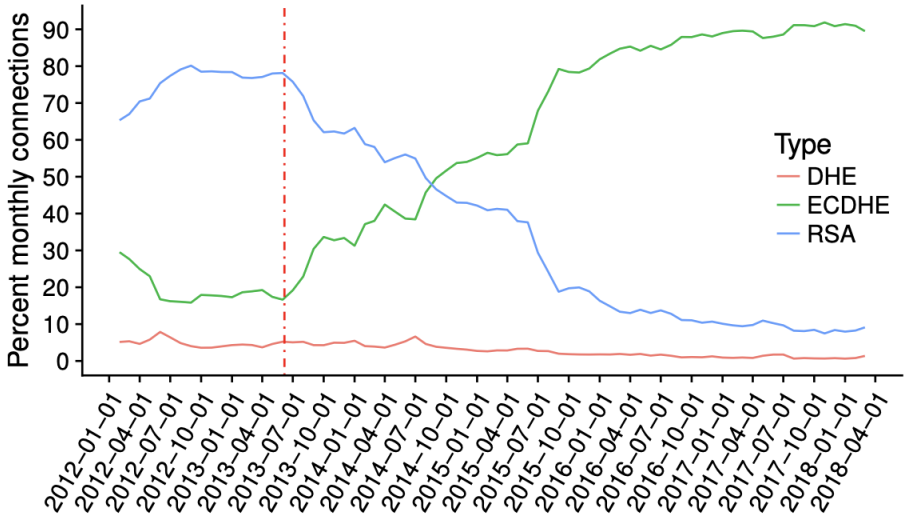
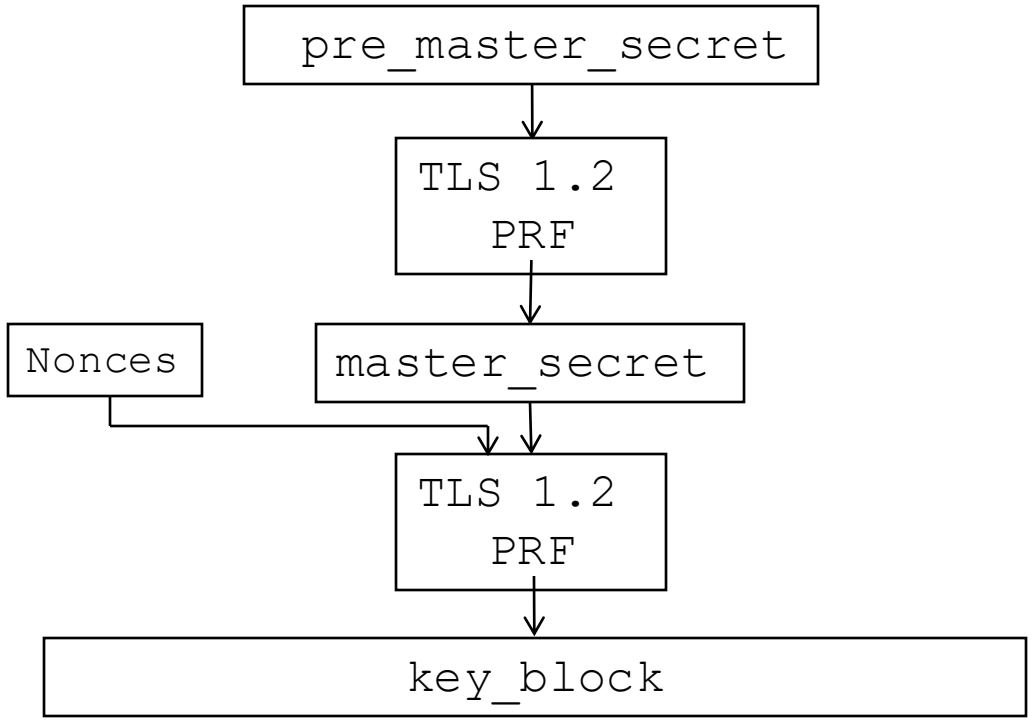



**Figure 8: Negotiated RSA and forward secret connections. Dotted line shows the date of first Snowden revelations.**



Source: Coming of Age: A Longitudinal Study of TLS Deployment, https://conferences.sigcomm.org/imc/2018/papers/imc18-final193.pdf

# Key Derivation in TLS v1.2



# Key Derivation in TLS v1.2

- Keys used by MAC and encryption (or AE) algorithms in the Record Protocol are derived from `pre_master_secret` (`pms`)
- Derive `master_secret` (`ms`) from `pms` using Pseudo-Random Function
- Default PRF for TLS1.2 is built by iterating HMAC-SHA256
- Derive `key_block` from `ms` and client/server nonces exchanged during Handshake Protocol using the TLS PRF in TLS v1.2
- `key_block` is split up into MAC keys, encryption keys and IVs for use in Record Protocol as needed
- NB: splitting up of `key_block` is cipher suite-dependent

# Authentication for ephemeral DH-based key establishment

**Client** | **Server**

`ClientHello (TLS_DHE_RSA_WITH_RC4_128_SHA)` →

← `ServerHello, Cert, ServerKeyExchange, ServerHelloDone`

$p$, $g$, $g^x$,
**RSAsig(nonces, params)**

1. Check Cert
2. Extract PubK from Cert
3. Use PubK to check server signature
4. Choose y, compute $g^y$, $(g^x)^y$

Server authenticated to client by showing its ability to sign client nonce using the Server's private key

`ClientKeyExchange:` $g^y$ →

`pre_master_secret:` $(g^y)^x$

← `ServerFinished`

# ClientFinished

**Client** | **Server**

ClientHello →

← ServerHello, Cert, [ServerKeyExchange,] ServerHelloDone

1. Derive ms
2. Compute ClientFinished = PRF(ms,transcript)

ClientKeyExchange, ClientFinished →

1. Derive ms
2. Compute ClientFinished' = PRF(ms,transcript)
3. Compare to received version

← ServerFinished

# `ClientFinished`

- Enables each side to check that both views of the protocol are the same
- Computed as `PRF(ms,transcript)`
  - `transcript` = sender's view of messages sent and received up to this point
- Compared by recipient to expected value
  - Protocol aborts if mismatch is observed
- Designed to prevent version rollback and cipher suite downgrade attacks
  - Of course, ineffective if attacker can compute `ms` during protocol run…

# Client authentication

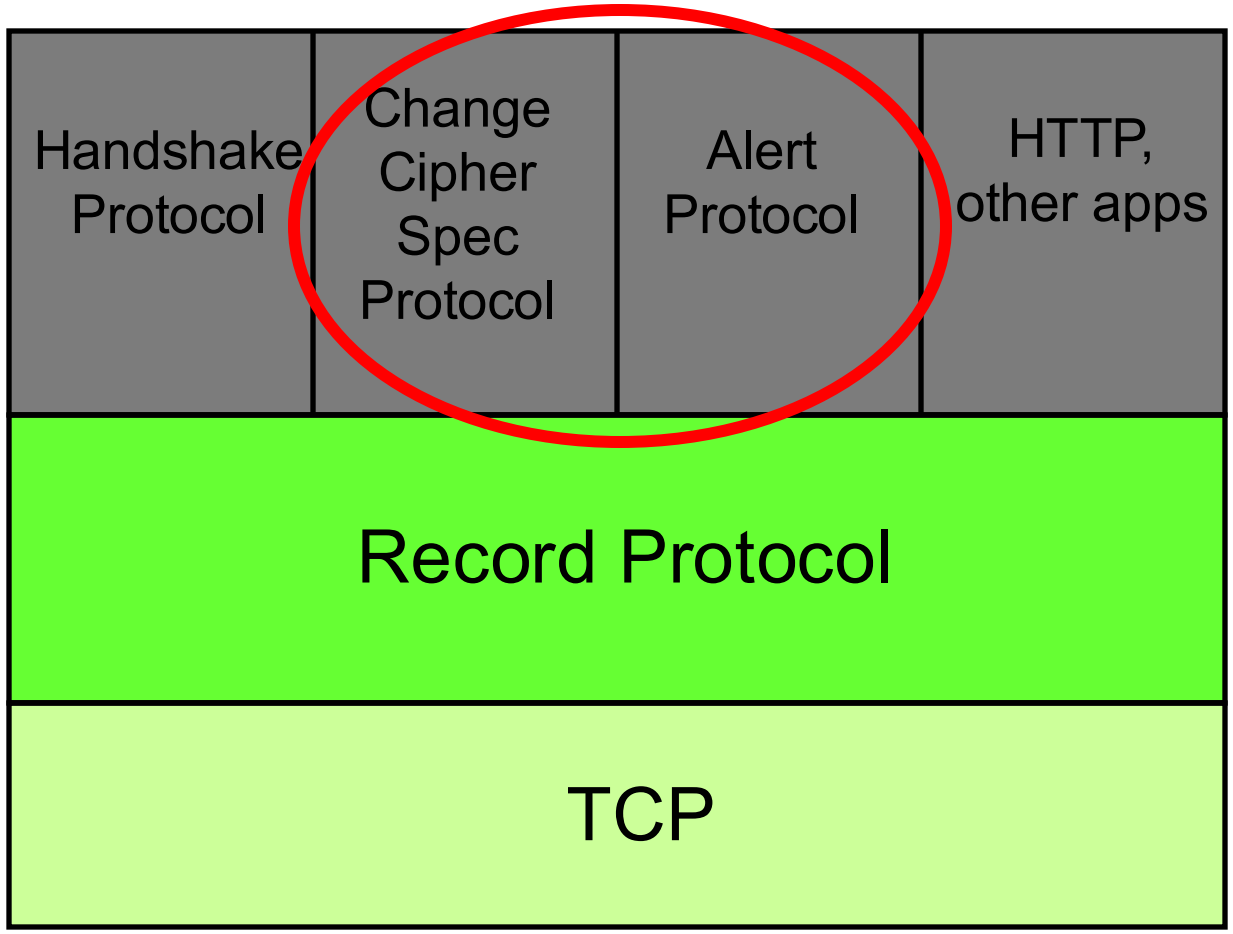
- Client authentication is optional and rarely used in the web setting
- Server requests client's certificate in its `Hello` message
- Client responds with:
  - `Cert`: client's certificate (chain)
  - `CertificateVerify`: signature on protocol transcript to this point

# Additional features of TLS Handshake

- Handshake Protocol also supports renegotiation and session resumption
- **Renegotiation** allows re-keying and change of cipher suite during a session
- E.g., to force strong client-side authentication before access to a particular resource on the server is allowed
- **Session resumption** allows authentication and shared secrets to be reused across multiple, parallel connections in a single session
- E.g., allows fetching multiple resources from same website without redoing full, expensive Handshake Protocol

# Other TLS protocols

# TLS Protocol Architecture



# Other TLS protocols

**Alert** protocol

- Management of SSL/TLS connections and sessions, error messages
- Fatal errors and warnings
- Defined actions to ensure clean session termination by both client and server

**Change Cipher Spec** protocol

- Used to indicate that entity is changing to recently agreed cipher suite

- Both run over **Record Protocol** and so are peers of the **Handshake Protocol**

# TLS is complex!

- Simplistic view of TLS:
  - Handshake Protocol followed by Record Protocol
- Reality is much more complex:
  - Initial Handshake Protocol over Record Protocol with no keys
  - Change Cipher Spec. Protocol message: switch on new keys
  - Completion of Handshake via exchange of `Finished` messages
  - Now running over keyed Record Protocol
  - May be arbitrary sequences of Session Resumption and Renegotiation runs
    Most of this activity is hidden from applications
- This complexity has turned out to have negative security consequences

# Homework

- Read Chapter 13 of "*Serious Cryptography*"

# Homework: play with https://tls12.xargs.org

## ❧ The Illustrated TLS 1.2 Connection ❧

### Every byte explained and reproduced

In this demonstration a client connects to a server, negotiates a TLS 1.2 session, sends "ping", receives "pong", and then terminates the session. Click below to begin exploring.

❯ Client Hello

❮ Server Hello

❮ Server Certificate

± Server Key Exchange Generation

❮ Server Key Exchange

# Homework: also play with https://tls13.xargs.org

## ❧ The Illustrated TLS 1.3 Connection ❧

### Every byte explained and reproduced

In this demonstration a client connects to a server, negotiates a TLS 1.3 session, sends "ping", receives "pong", and then terminates the session. Click below to begin exploring.

#### ± Client Key Exchange Generation ×

The client begins by generating a private/public keypair for key exchange. Key exchange is a technique where two parties can agree on the same number without an eavesdropper being able to tell what the number is.

An explanation of the key exchange can be found on my X25519 site, but doesn't need to be understood in depth for the rest of this page.

The private key is chosen by selecting an integer between 0 and $2^{256}$-1. The client does this by generating 32 bytes (256 bits) of random data. The private key selected is:

```
202122232425262728292a2b2c2d2e2f303132333435363738393a3b3c3d3e3f
```